



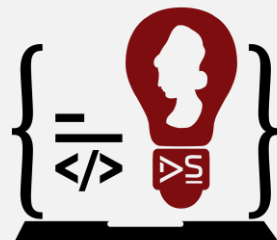
CICLO 1/ 2025 - ASIGNATURA:

LÓGICA DE PROGRAMACIÓN

UNIVERSIDAD DE EL SALVADOR - FMOcc

Departamento de Ingeniería y Arquitectura

Docente: _____



Ingeniería en Desarrollo
de Software

```
static bool init(CURL *conn, char *url, struct http_header  
{  
    CURLcode code; struct http_header header;  
  
    conn = curl_easy_init();  
  
    if (conn == NULL)  
    {  
        fprintf(stderr, "Failed to create CURL connection\n");  
        exit(EXIT_FAILURE);  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_ERRORBUFFER,  
errorBuffer);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set error buffer [%d]\n",  
code);  
        return false;  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_URL, url);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set URL [%s]\n", errorBuff-  
code);  
        return false;  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_FOLLOWLOCATION,  
_OK);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set redirect option [%s]\n",  
code);  
        return false;  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_WRITEFUNCTION,  
writer);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set writer [%s]\n",  
errorBuffer);  
        return false;  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_WRITEDATA,  
&buffer);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set write data [%s]\n",  
errorBuffer);  
        return false;  
    }  
  
    return true;  
}
```

Avisos y Anuncios semana 2

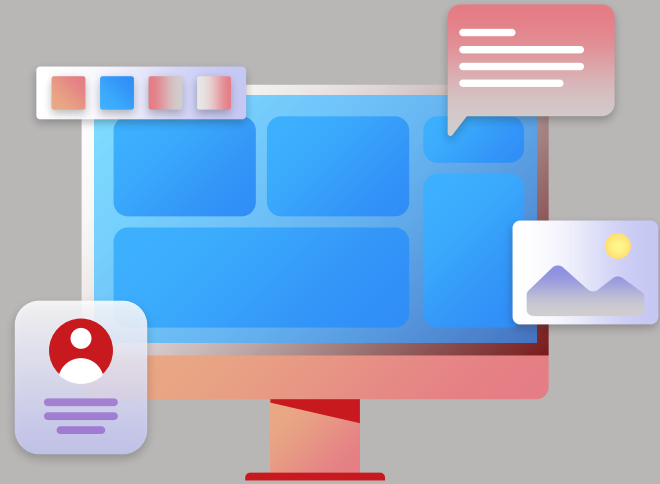


- Actividad a Desarrollar
 - Desarrollar Guía de laboratorio
 - Participación en el Foro
 - Leer los contenidos de esta semana



Funciones, Arreglos, Estructuras, Memoria, Archivos, Listas

Unidad 1



Agenda.

/1.1.4 Funciones como
Parámetros

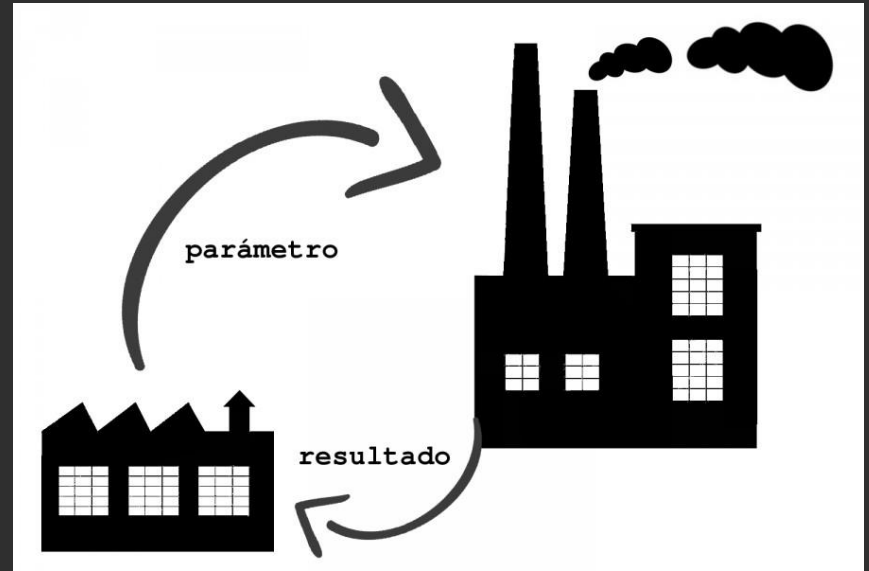
/1.2.1 Definición de
Arreglos Con
Punteros

/1.2.2 Punteros a
Cadenas

/1.2.3 Aritmética de
Punteros



1.1.4 Funciones como Parámetros



Funciones Como Parámetros



Pasar una función como parámetro en C/C++, existen diferentes formas de diseñar funciones que acepten otra función como argumento.

Funciones Como Parámetros

Pasando puntero a una función: Una función se puede pasar a otra función pasando su dirección a esa función.

```
// Function that takes a pointer  
// to a function  
int invoke(int x, int y,  
           int (*func)(int, int))  
{  
    return func(x, y);  
}
```



```
// Driver Code  
int main()  
{  
    // Pass pointers to add & multiply  
    // function as required  
    cout << "Addition of 20 and 10 is ";  
    cout << invoke(20, 10, &add)  
           << '\n';  
  
    cout << "Multiplication of 20"  
           << " and 10 is ";  
    cout << invoke(20, 10, &multiply)  
           << '\n';  
  
    return 0;  
}
```

Funciones Como Parámetros



Usando `std::function<>`: En C++, hay una clase de plantilla `std::function<>` que permite pasar funciones como parámetros..

.

Funciones Como Parámetros

Usando std::function

```
// C++ program to illustrate the passing
// of functions as an object parameter
#include <functional>
#include <iostream>
using namespace std;

// Define add and multiply to
// return respective values
int add(int x, int y)
{
    return x + y;
}

int multiply(int x, int y)
{
    return x * y;
}
```

```
// Function that accepts an object of
// type std::function<> as a parameter
// as well
int invoke(int x, int y,
          function<int(int, int)> func)
{
    return func(x, y);
}
```

```
// Driver code
int main()
{
    // Pass the required function as
    // parameter using its name
    cout << "Addition of 20 and 10 is ";
    cout << invoke(20, 10, &add)
          << '\n';

    cout << "Multiplication of 20"
          << " and 10 is ";
    cout << invoke(20, 10, &multiply)
          << '\n';

    return 0;
}
```

Funciones Como Parámetros



Utilizando Lambdas: En C++ proporciona una forma de definir objetos de función anónimos, únicos y en línea. Estas lambdas se pueden definir en un lugar donde se requiere pasar una función como parámetro

```
// C++ program to pass the function as
// parameter as a lambda expression
#include <functional>
#include <iostream>
using namespace std;

// Function that takes a pointer
// to a function
int invoke(int x, int y,
          function<int(int, int)> func)
{
    return func(x, y);
}
```

```
// Driver Code
int main()
{
```

```
    cout << "Addition of 20 and 10 is ";
    int k = invoke(20, 10,
        [](int x,
           int y) -> int {
                return x + y;
            });

    cout << k << '\n';

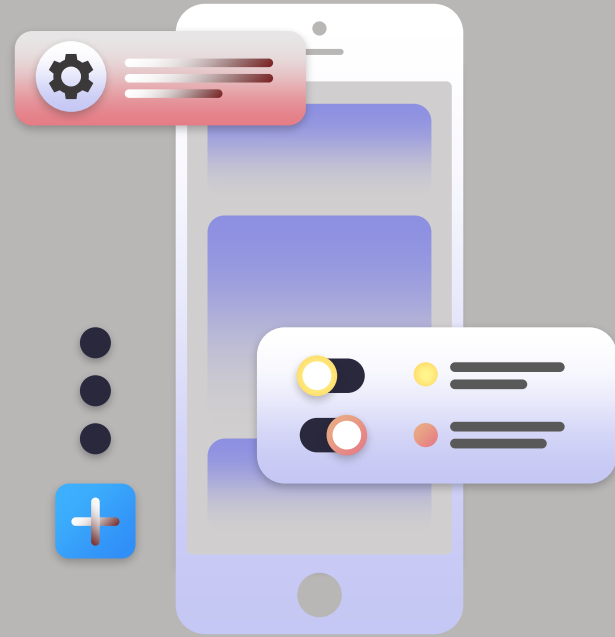
    // Perform Multiplication
    cout << "Multiplication of 20"
        << " and 10 is ";
    int l = invoke(20, 10,
        [](int x,
           int y) -> int {
                return x * y;
            });

    cout << l << '\n';

    return 0;
}
```

1.2

Arreglos y Punteros





/INTRODUCCIÓN

- Arreglos: Un array o arreglo es una secuencia de objetos del mismo tipo.
- Un puntero es una dirección de memoria. El concepto de punteros tiene correspondencia en la vida diaria. Cuando se envía una carta por correo, su información se entrega basada en un puntero que es la dirección de esa carta

1.2.1 Definición de Arreglos Con Punteros



Arreglos con Punteros

Los punteros o apuntadores son, por definición, tan sólo variables usadas para almacenar direcciones, también existe una relación directa y estrecha entre los nombres de los arreglos y los punteros, ya que, un nombre de un array es simplemente un puntero.

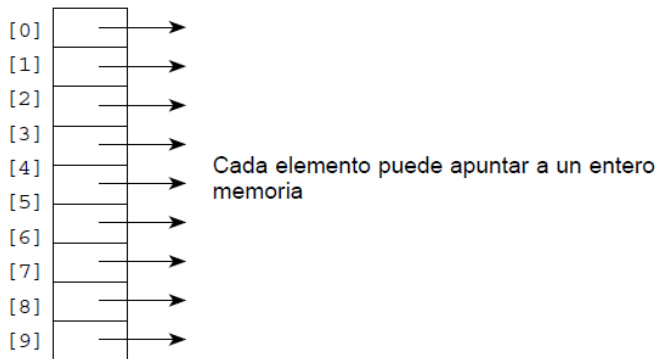
```
int lista[5] = {10, 20, 30, 40, 50};
```

	Memoria	
lista → [0]	10	← lista
[1]	20	← *(lista+1)
[2]	30	← *(lista+2)
[3]	40	← *(lista+3)
[4]	50	← *(lista+4)

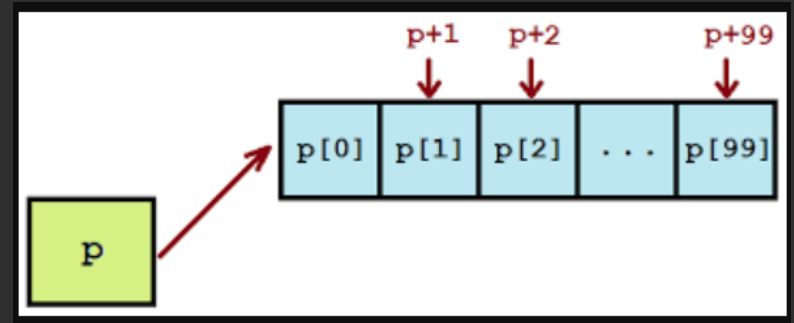
Arreglos con Punteros

`int *ptr[10];` // reserva un array de 10 punteros a enteros
La imagen de abajo muestra cómo C++ organiza este array. Cada elemento contiene una dirección que apunta a otros valores de la memoria

```
ptr[5] = &edad    // ptr[5] apunta a la dirección de edad
```



1.2.2 Punteros a Cadenas



Punteros a Cadenas



La inicialización de un array de punteros a cadenas se puede realizar con una declaración similar a ésta

```
char *nombres_meses[12] = { "Enero", "Febrero", "Marzo",  
"Abril", "Mayo", "Junio", "Julio", "Agosto", "Septiembre",  
"Octubre", "Noviembre", "Diciembre" };
```

Punteros a Cadenas



Los punteros se pueden utilizar en lugar de índices de arrays.

Es posible, entonces, considerar dos tipos de definiciones de cadena:

```
char cadena1[]="Hola viejo mundo"; //array contiene cadena  
char *cptr = "C++ a su alcance"; //un puntero a la cadena
```

1.2.3 Aritmética de Punteros



Aritmética de Punteros.



Al contrario que un nombre de array, que es un puntero constante y no se puede modificar. Un puntero es un puntero variable que se puede modificar. Como consecuencia, se pueden realizar ciertas operaciones aritméticas sobre punteros.

Operaciones No válidas con punteros

- No se pueden sumar dos punteros.
- No se pueden multiplicar dos punteros.
- No se pueden dividir dos punteros.

Aritmética de Punteros.



Los operadores de incremento y decremento pueden aplicarse como operadores de apuntador de prefijo o posfijo. Todas las siguientes combinaciones que usan apuntadores son válidas:

```
*ptNum++    // usa el apuntador y luego lo incrementa
*++ptNum    | // incrementa el apuntador antes de usarlo
*ptNum--    // usa el apuntador y luego lo decrementa
*--ptNum    // decrementa el apuntador antes de usarlo
```

Aritmética de Punteros.



Además, las direcciones en los apuntadores pueden compararse usando cualquiera de los operadores relacionales (`==`, `!=`, `<`, `>`, etc.) que son válidos para comparar otras variables. Al realizar aritmética en los apuntadores debe tenerse cuidado de producir direcciones que apunten a algo significativo. Al comparar apuntadores también deben hacerse comparaciones que tengan sentido.

Ejemplo Aritmética de Punteros.

```
#include <iostream>
using namespace std;

int main()
{
    const int NUMS = 5;

    int nums[NUMS] = {16, 54, 7, 43, -5};
    int i, total = 0, *nPt;

    nPt = nums;    // almacena la dirección de nums[0] en nPt
    for (i = 0; i < NUMS; i++)
        total = total + *nPt++;
    cout << "El total de los elementos del arreglo es " << total << endl;

    return 0;
}
```

/Recordatorios

/Realizar la actividad evaluada de la semana

/Leer los contenidos vistos en clase



/Participar en foros

/Codifica los ejemplos expuestos

/GRACIAS!

/ALGUNA PREGUNTA?

